

BACKTRACKING-

1. Pick / Not Pick Pattern

Har element ke liye 2 choices:

- Include karo
- Exclude karo

Questions

- Subsets

```
void sub(vector<int>& nums, int idx, vector<int>& temp, vector<vector<int>>& ans){
    if(idx==nums.size()){
        ans.push_back(temp);
        return;
    }
    temp.push_back(nums[idx]);
    sub(nums, idx+1, temp, ans);
    temp.pop_back();
    sub(nums, idx+1, temp, ans);
}

vector<vector<int>> subsets(vector<int>& nums) {
    vector<int>temp;
    vector<vector<int>>ans;
    sub(nums,0,temp,ans);
    return ans;
};
```

- Subsets II

```
void sub(vector<int>& nums, int idx,
        vector<int>& temp,
        vector<vector<int>>& ans) {
    if(idx == nums.size()) {
        ans.push_back(temp);
        return;
    }
    // include current element
    temp.push_back(nums[idx]);
    sub(nums, idx + 1, temp, ans);
    temp.pop_back();
    // skip duplicates while excluding
    while(idx + 1 < nums.size() &&
          nums[idx] == nums[idx + 1]) {
        idx++;
    }
    // exclude current element
    sub(nums, idx + 1, temp, ans);
}

vector<vector<int>> subsetsWithDup(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    vector<int>temp;
    vector<vector<int>>ans;
    sub(nums,0,temp,ans);
    return ans;
}
```

- Combination Sum

```

void combine(vector<int>& candidates,int target,vector<int>& temp,vector<vector<int>>& ans,int idx){
    if(target==0){
        ans.push_back(temp);
        return;
    }
    for(int i=idx;i<candidates.size();i++){
        if(candidates[i]>target) continue;
        temp.push_back(candidates[i]);
        combine(candidates,target-candidates[i],temp,ans,i);
        temp.pop_back();
    }
}

vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
    vector<int>temp;
    vector<vector<int>>ans;
    sort(candidates.begin(),candidates.end());
    // int sum=0;
    combine(candidates,target,temp,ans,0);
    return ans;
}

```

- Combination Sum II

```

8 public:
7 void combine(vector<int>& candidates,int target,vector<int>& temp,vector<vector<int>>& ans,int idx){
6     if(target==0){
5         ans.push_back(temp);
4         return;
3     }
2     for(int i=idx;i<candidates.size();i++){
1         if(i>idx&&candidates[i]==candidates[i-1]) continue;
0         if(candidates[i]>target) continue;
9         temp.push_back(candidates[i]);
8         combine(candidates,target-candidates[i],temp,ans,i+1);
7         temp.pop_back();
6     }
5 }
4 vector<vector<int>> combinationSum2(vector<int>& candidates, int target) {
3     vector<int>temp;
2     vector<vector<int>>ans;
1     sort(candidates.begin(),candidates.end());
    combine(candidates,target,temp,ans,0);
    return ans;
}
}

```

- Target Sum

```

public:
int targetsum(vector<int>& nums,int target,int sum,int idx){
    if(idx==nums.size()){
        return (sum==target);
    }
    int take=targetsum(nums,target,sum+nums[idx],idx+1);
    int not_take=targetsum(nums,target,sum-nums[idx],idx+1);
    return take+not_take;
}

int findTargetSumWays(vector<int>& nums, int target) {
    int sum=0;
    return targetsum(nums,target,0,0);
}

```

Template:

```

void solve(int idx){
    if(idx==n){
        ans.push_back(temp);
        return;
    }

    // Pick
    temp.push_back(nums[idx]);
    solve(idx+1);
    temp.pop_back();

    // Not Pick
    solve(idx+1);
}

```

2. For Loop Backtracking Pattern

Current position se saare options try karo.

Questions

- Combination Sum

Letter Combinations of a Phone Number - Backtracking ke context me string ko vector of characters ki tarah treat kar sakte ho.

```
9 | unordered_map<int,string>mp={
8 | {'2',"abc"},
7 | {'3',"def"},
6 | {'4',"ghi"},
5 | {'5',"jkl"},
4 | {'6',"mno"},
3 | {'7',"pqrs"},
2 | {'8',"tuv"},
1 | {'9',"wxyz"}
2 | };
1 | void backtrack(string digits,int idx,string& st,vector<string>& result){
2 |     if(idx==digits.size()){
3 |         result.push_back(st);
4 |         return;
5 |     }
6 |     auto dig=digits[idx];
7 |     auto letter=mp[dig];
8 |     for(auto x:letter){
9 |         st.push_back(x);
10 |         backtrack(digits,idx+1,st,result);
11 |         st.pop_back();
12 |     }
13 | };
14 | vector<string> letterCombinations(string digits) {
15 |     string st="";
16 |     vector<string>result;
17 |     backtrack(digits,0,st,result);
18 |     return result;
}
```

- Restore IP Addresses

```
bool isValid(string str){
    if(str.size() > 1 && str[0] == '0')
        return false;
    int val=stoi(str);
    return val<=255;
}

void address(string& s,string path,int part,vector<string>& result,int idx){
    if(idx==s.length()&&part==4){
        path.pop_back();
        result.push_back(path);
        return;
    }
    if(idx+1<=s.length()){
        address(s,path+s.substr(idx,1)+".",part+1,result,idx+1);
    }

    if(idx+2<=s.length()&&isValid(s.substr(idx,2))){
        address(s,path+s.substr(idx,2)+".",part+1,result,idx+2);
    }

    if(idx+3<=s.length()&&isValid(s.substr(idx,3))) {
        address(s,path+s.substr(idx,3)+".",part+1,result,idx+3);
    }
}

vector<string> restoreIpAddresses(string s) {
    if(s.length()>12) return {};
    string path="";
    int part=0;
    vector<string>result;
    address(s,path,0,result,0);
    return result;
}
```

- Palindrome Partitioning

```

26 public:
25 bool ispalindrome(string& s,int l,int r){
24     while(l<r){
23         if(s[l]!=s[r]){
22             return false;
21         }
20         l++;
19         r--;
18     }
17     return true;
16 }
15 void backtrack(string& s,int idx,vector<string>& temp,vector<vector<string>>& result){
14     if(idx==s.length()){
13         result.push_back(temp);
12         return;
11     }
10     for(int i=idx;i<s.length();i++){
9         if(ispalindrome(s,idx,i)){
8             temp.push_back(s.substr(idx,i-idx+1));
7             backtrack(s,i+1,temp,result);
6             temp.pop_back();
5         }
4     }
3 }
2 vector<vector<string>> partition(string s) {
1     vector<string>temp;
0     vector<vector<string>>&result;
1     backtrack(s,0,temp,result);
2     return result;

```

s = "aab"

Aur maan lo:

↗ C++

```
idx = 0
i = 1
```

To line:

↗ C++

```
temp.push_back(s.substr(idx, i-idx+1));
```

becomes:

↗ C++

```
temp.push_back(s.substr(0, 1-0+1));
```

↗ C++

```
temp.push_back(s.substr(0, 2));
```

substr(start, length) hota hai.

- Subset 2

```

3 void sub(vector<int>& nums,int idx,vector<int>& temp,vector<vector<int>>& an
4     ans.push_back(temp);
5     unordered_set<int>s1;
6     for(int i=idx;i<nums.size();i++){
7         if(s1.find(nums[i])!=s1.end()){
8             continue;
9         }
10        s1.insert(nums[i]);
11        temp.push_back(nums[i]);
12        sub(nums,i+1,temp,ans);
13        temp.pop_back();
14        // sub(nums,i+1,temp,ans);
15    }
16 }
17 vector<vector<int>> subsetsWithDup(vector<int>& nums) {
18     sort(nums.begin(),nums.end());
19     vector<int>temp;
20     vector<vector<int>>&ans;
21     sub(nums,0,temp,ans);
22     return ans;

```

Template:

```

void solve(int start){
    ans.push_back(temp);

    for(int i=start;i<n;i++){
        temp.push_back(nums[i]);
        solve(i+1);
        temp.pop_back();
    }
}

```

3. Permutation Pattern (Visited Array)

Har level par koi bhi unused element choose kar sakte ho.

Questions

- Permutations

```

void perm(vector<int>& nums, vector<vector<int>>& result, int idx){
    if(idx==nums.size()){
        result.push_back(nums);
        return;
    }
    for(int i=idx; i<nums.size(); i++){
        swap(nums[i], nums[idx]);
        perm(nums, result, idx+1);
        swap(nums[i], nums[idx]);
    }
}

vector<vector<int>> permute(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    vector<vector<int>> result;
    perm(nums, result, 0);
    return result;
}
};

```

- Permutations II

```

12 void perm(vector<int>& nums, vector<vector<int>>& result, int idx){
11     if(idx==nums.size()){
10         result.push_back(nums);
9         return;
8     }
7     unordered_set<int> s1;
6     for(int i=idx; i<nums.size(); i++){
5         if(s1.find(nums[i])!=s1.end()){
4             continue;
3         }
2         s1.insert(nums[i]);
1         swap(nums[i], nums[idx]);
;         perm(nums, result, idx+1);
1         swap(nums[i], nums[idx]);
2     }
3 }
4 }

5 vector<vector<int>> permuteUnique(vector<int>& nums) {
6     sort(nums.begin(), nums.end());
7     // vector<int> temp;
8     vector<vector<int>> result;
9     perm(nums, result, 0);
10    return result;

```

Template:

```

void solve() {
    if(temp.size()==n){
        ans.push_back(temp);
        return;
    }

    for(int i=0; i<n; i++){
        if(vis[i]) continue;

        vis[i]=true;
        temp.push_back(nums[i]);

        solve();

        temp.pop_back();
        vis[i]=false;
    }
}

```

4. Swap Based Permutation Pattern

Array ko inplace modify karte hain.

Questions

- Permutations

```

void perm(vector<int>& nums, vector<vector<int>>& result, int idx){
    if(idx==nums.size()){
        result.push_back(nums);
        return;
    }
    for(int i=idx; i<nums.size(); i++){
        swap(nums[i], nums[idx]);
        perm(nums, result, idx+1);
        swap(nums[i], nums[idx]);
    }
}

vector<vector<int>> permute(vector<int>& nums) {
    sort(nums.begin(), nums.end());
    vector<vector<int>> result;
    perm(nums, result, 0);
    return result;
}
};

```

- String Permutations

Template:

```

void solve(int idx){
    if(idx==n){
        ans.push_back(nums);
        return;
    }

    for(int i=idx; i<n; i++){
        swap(nums[idx], nums[i]);

        solve(idx+1);

        swap(nums[idx], nums[i]);
    }
}

```

5. Grid Backtracking Pattern

4 directions ya 8 directions mein move karna.

Questions

- Word Search
- Number of Islands
- Rat in a Maze

Template:

```

bool dfs(int r, int c){
    if(found) return true;

    vis[r][c]=1;

    for(auto dir:dirs){
        int nr=r+dir[0];
        int nc=c+dir[1];

        if(valid(nr, nc) && !vis[nr][nc]){
            dfs(nr, nc);
        }
    }

    vis[r][c]=0;
}

```

6. Constraint Backtracking Pattern

Har choice valid hai ya nahi pehle check karte hain.

Questions

- N-Queens
- Sudoku Solver

Template:

```
for (each choice) {  
    if (isValid(choice)) {  
        place(choice);  
        solve(next);  
        remove(choice);  
    }  
}
```

Recommended Order

1. Subsets -done
2. Subsets II -done
3. Combination Sum -done
4. Combination Sum II -done
5. Letter Combinations of a Phone Number -done
6. Palindrome Partitioning -done
7. Permutations -done
8. Permutations II -done
9. Word Search
10. N-Queens
11. Sudoku Solver - board[3*(row/3)+i/3][3*(col/3)+i%3]

```
bool issudokuvalid(vector<vector<char>>& board, int row, int col, char ch) {  
    for (int i = 0; i < 9; i++) {  
        if (board[i][col] == ch || board[row][i] == ch ||  
            board[3*(row/3)+i/3][3*(col/3)+i%3] == ch) {  
            return false;  
        }  
    }  
    return true;  
}  
  
bool solve(vector<vector<char>>& board) {  
    for (int i = 0; i < board.size(); i++) {  
        for (int j = 0; j < board.size(); j++) {  
            if (board[i][j] == '.') {  
                for (int ch = '1'; ch <= '9'; ch++) {  
                    if (issudokuvalid(board, i, j, ch)) {  
                        board[i][j] = ch;  
                        if (solve(board)) {  
                            return true;  
                        }  
                    }  
                    else {  
                        board[i][j] = '.';  
                    }  
                }  
            }  
        }  
        return false;  
    }  
    return true;  
}  
  
void solveSudoku(vector<vector<char>>& board) {  
    solve(board);  
}
```

Beautiful Arrangement

```
public:
void func(vector<int>& nums,int idx,vector<vector<int>>& result){
    if(idx==nums.size()){
        result.push_back(nums);
        return;
    }
    for(int i=idx;i<nums.size();i++){
        if((idx+1)%nums[i]==0||nums[i]%(idx+1)==0){
            swap(nums[i],nums[idx]);
            func(nums,idx+1,result);
            swap(nums[i],nums[idx]);
        }
    }
}

int countArrangement(int n) {
    vector<int>nums;
    for(int i=1;i<=n;i++){
        nums.push_back(i);
    }
    vector<vector<int>>result;
    func(nums,0,result);
    return result.size();
}
```

Sum of All Subset XOR Totals

```
public:
void func(vector<int>& nums,int& sum,int idx,vector<int>& temp){
    if(idx==nums.size()){
        int xorr=0;
        for(auto x:temp){
            xorr^=x;
        }
        sum+=xorr;
        return;
    }
    temp.push_back(nums[idx]);
    func(nums,sum,idx+1,temp);
    temp.pop_back();
    func(nums,sum,idx+1,temp);
}

int subsetXORSum(vector<int>& nums) {
    int sum=0;
    vector<int>temp;
    func(nums,sum,0,temp);
    return sum;
}
```

Find Unique Binary String

```

3 bool isunique(string& curr,vector<string>& nums){
2     for(auto x:nums){
1         if(x==curr){
0             return false;
9         }
8     }
7     return true;
6 }
5 void func(vector<string>& nums,string& curr,vector<string>& result){
4     if(curr.size()==nums[0].size()){
3         if(isunique(curr,nums)){
2             result.push_back(curr);
1         }
        return;
    }
    curr.push_back('1');
    func(nums,curr,result);
    curr.pop_back();
    curr.push_back('0');
    func(nums,curr,result);
    curr.pop_back();
}
0 string findDifferentBinaryString(vector<string>& nums) {
1     string curr="";
2     vector<string>result;
3     func(nums,curr,result);
4     return result[0];

```

Generate Binary Strings Without Adjacent Zeros

```

--
9 bool adjacent(string& curr){
8     for(int i=1;i<curr.length();i++){
7         if(curr[i]=='0'&&curr[i-1]=='0'){
6             return false;
5         }
4     }
3     return true;
2 }
1 void func(int n,string& curr,vector<string>& result){
12     if(n==curr.size()){
1         if(adjacent(curr)){
2             result.push_back(curr);
3         }
4         return;
5     }
6     curr.push_back('1');
7     func(n,curr,result);
8     curr.pop_back();
9     curr.push_back('0');
10    func(n,curr,result);
11    curr.pop_back();
12 }
14 vector<string> validStrings(int n) {
15     string curr="";
16     vector<string>result;
17     func(n,curr,result);
18     return result;

```

The k-th Lexicographical String of All Happy Strings of Length n

```

void solve(int n,vector<string>& ans,string& temp,int k){
    if(temp.size()==n){
        ans.push_back(temp);
        return;
    }
    for(auto x:{"a","b","c"}){
        if(temp.empty()||temp.back()!=x){
            temp.push_back(x);
            solve(n,ans,temp,k);
            temp.pop_back();
        }
    }
}
string getHappyString(int n, int k) {
    string temp="";
    vector<string>ans;
    solve(n,ans,temp,k);
    if(k>ans.size()) return "";
    return ans[k-1];
}

```

Letter Case Permutation

```

void permutation(string& s,int idx,string& temp,vector<string>& result){
    if(idx==s.size()){
        result.push_back(temp);
        return;
    }
    auto ch=s[idx];
    if(isdigit(ch)){
        temp.push_back(ch);
        permutation(s,idx+1,temp,result);
        temp.pop_back();
    }
    else {
        for(auto x:{toupper(ch),tolower(ch)}){
            temp.push_back(x);
            permutation(s,idx+1,temp,result);
            temp.pop_back();
        }
        /* temp.push_back(toupper(ch));
        permutation(s,idx+1,temp,result);
        temp.pop_back();

        temp.push_back(tolower(ch));
        permutation(s,idx+1,temp,result);
        temp.pop_back();
        */
    }
}

vector<string> letterCasePermutation(string s) {
    string temp="";
    vector<string>result;
    permutation(s,0,temp,result);
    return result;
}

```

Generate Parentheses

```

bool isValid(string& temp){
    stack<char>st;
    for(auto x:temp){
        if(x=='('){
            st.push('(');
        }
        else if(!st.empty()&&st.top()=='('){
            st.pop();
        }
        else{
            return false;
        }
    }
    return st.empty();
}

void func(int n,string& temp,vector<string>& result){
    if(n*2==temp.size()){
        if(isValid(temp)){
            result.push_back(temp);
        }
        return;
    }
    temp.push_back('(');
    func(n,temp,result);
    temp.pop_back();

    temp.push_back(')');
    func(n,temp,result);
    temp.pop_back();
}

vector<string> generateParenthesis(int n) {
    vector<string>result;
    string temp="";
    func(n,temp,result);
    return result;
}

```

Combination Sum III

```

void func(int k,int n,vector<int>& temp,vector<vector<int>>& result,int sum,int idx){
    if(sum==n&&temp.size()==k){
        result.push_back(temp);
        return;
    }
    for(int i=idx;i<=9;i++){
        temp.push_back(i);
        func(k,n,temp,result,sum+i,i+1);
        temp.pop_back();
    }
}

vector<vector<int>> combinationSum3(int k, int n) {
    vector<int>temp;
    vector<vector<int>>result;
    if(k>n) return result;
    int sum=0;
    func(k,n,temp,result,sum,1);
    return result;
}

```

Letter Tile Possibilities

```
1  -----
2  void perm(string& tiles,int& count,int idx){
3      if(idx==tiles.size()){
4          return;
5      }
6      unordered_set<int>unseen;
7      for(int i=idx;i<tiles.size();i++){
8          if(unseen.find(tiles[i])!=unseen.end()){
9              continue;
10         }
11         unseen.insert(tiles[i]);
12         swap(tiles[i],tiles[idx]);
13         count++;
14         perm(tiles,count,idx+1);
15         swap(tiles[i],tiles[idx]);
16     }
17 }
18
19 int numTilePossibilities(string tiles) {
20     sort(tiles.begin(),tiles.end());
21     int count=0;
22     perm(tiles,count,0);
23     return count;
24 }
```